

Faculty of Engineering and Technology
Object Oriented Programming (CEUE203)
A.Y 2026 – 27 Semester – 3
Practical List

Practical Number		CO/PO
5	Part A — Practice Programs (Hour 1) Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Shape areas: an abstract Shape with abstract area(); subclasses Circle, Rectangle, Triangle. Put a mix in a Shape[] and loop once to print each area and the running total, tracking the largest — one loop handling every shape through polymorphism. 2. Payroll: an abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime (hours × rate) and Intern (stipend), each calling super for the shared name/id. Loop a mixed array to print each salary and the total; use instanceof to add a note only for interns. 3. (For Advanced Learners – home) Model a few media types that each compute a late fee differently and total the fees for a returned batch. Design the hierarchy and rules yourself. Part B — Project Work (Hour 2): MiniBank A real bank has different account types. Building on the single Account class, this lab makes Account abstract and adds three subclasses — SavingsAccount, CurrentAccount and FixedDepositAccount — each with its own interest and withdrawal rules chosen automatically by polymorphism. This lets the rest of MiniBank treat every account through one common type. 1. Make Account an abstract class. Add two abstract methods: double interestRate() and boolean canWithdraw(long amount). 2. Create a class SavingsAccount that extends Account with a long field minBalance; interestRate() returns 4.0; canWithdraw() returns true only if the balance stays greater than or equal to minBalance. 3. Create a class CurrentAccount that extends Account with a long field overdraftLimit; interestRate() returns 0; canWithdraw() allows the balance to fall to minus overdraftLimit. 4. Create a class FixedDepositAccount that extends Account; interestRate() returns 7.0; canWithdraw() returns false (the deposit is locked). 5. Call super(...) from each subclass constructor to initialise the inherited fields. 6. In main, place objects of the three types in an Account[] array, loop through it and call interestRate() on each (this demonstrates run-time polymorphism); use a pattern instanceof check to handle one type specially. Key Questions / Analysis / Interpretation to be evaluated during/after Implementation 1. What is the difference between an abstract class and a concrete class? 2. What is dynamic method dispatch (run-time polymorphism)? 3. What does the super keyword do in a subclass constructor? Supplementary Problems – • Add a method monthlyInterest() in Account that uses interestRate() and balance. • Override canWithdraw() in FixedDepositAccount to allow withdrawal after a maturity date. Key Skills to be addressed – Inheritance with extends; abstract classes and methods; method overriding; run-time polymorphism; super; object casting; pattern matching with instanceof. Applications – Type hierarchies (account types, employee types, shape types) are used wherever many variants share common behaviour. Learning Outcome – The student can design a class hierarchy and use polymorphism to write extensible code. (CO1, CO2) Dataset / Test Data (Source and Description, if applicable): No external dataset. One object of each account type is created in code.	CO1, CO2 PO1, PO2, PO3

Practical Number		CO/PO
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git. • Total Hours of Problem Definition Implementation: 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • Total Hours of Engagement = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours. Post Laboratory Work Description: Commit and tag lab-05. Confirm that calling <code>interestRate()</code> on the array prints the correct rate for each subtype. Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct abstract base and subclasses – 40%; polymorphic behaviour – 25%; super and instanceof – 15%; naming conventions and viva – 20%. Expected Input / Process / Output Input: In code: a <code>SavingsAccount</code>, a <code>CurrentAccount</code> and a <code>FixedDepositAccount</code> placed in an <code>Account[]</code> array. Process: The loop calls <code>interestRate()</code> on each element; Java selects the subclass version at run time (polymorphism). Expected Output: The program prints 4.0 for the savings account, 0.0 for the current account and 7.0 for the fixed deposit.	